



UNIVERSIDAD DEL BÍO-BÍO

FACULTAD DE CIENCIAS EMPRESARIALES

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN Y
TECNOLOGÍA DE LA INFORMACIÓN

INGENIERÍA CIVIL EN INFORMÁTICA

PROYECTO DE AUTÓMATAS

Implementación de Técnicas asociadas a Lenguajes Regulares

Integrantes:

Matías Constanzo Monsalve
Benjamin Valenzuela Muñoz

Profesor:

Luis Gajardo Diaz

Fecha de Entrega:

09 de mayo de 2026

Índice

1. Descripción del Programa	2
1.1. Objetivo General	2
1.2. Funcionalidades Implementadas	2
1.3. Arquitectura del Proyecto	3
1.4. Formato de Archivo de Entrada	3
2. Guía Rápida de Uso	4
2.1. Pasos para Ejecutar el Programa	4
2.2. Ejemplo Completo de Ejecución	4
3. Requisitos del Sistema	5
3.1. Software Necesario	5
3.2. Instalación de GraphViz	5
4. Ejemplos de Uso	5
4.1. Ejemplo 1: Dos AFD Equivalentes	5
4.1.1. Archivos de Entrada	5
4.1.2. Ejecución	6
4.1.3. Salida Esperada	6
4.2. Ejemplo 2: Conversión de AFND a AFD	7
4.2.1. Archivos de Entrada	7
4.2.2. Ejecución	8
4.2.3. Salida Esperada (Resumen)	8
4.3. Ejemplo 3: Autómatas No Equivalentes	9
4.3.1. Archivos de Entrada	9
4.3.2. Ejecución	9
4.3.3. Salida Esperada	9
5. Algoritmos Implementados	9
5.1. Conversión AFND $\rightarrow$ AFD (Subset Construction)	9
5.2. Minimización de AFD (Algoritmo de Partición)	10
5.3. Verificación de Equivalencia (BFS sobre pares)	10
6. Visualización con GraphViz	11
6.1. Archivos Generados	11
6.2. Formato DOT	11
6.3. Generación Manual de Imágenes	12

1 Descripción del Programa

1.1 Objetivo General

Desarrollar una aplicación en Java que permita procesar, transformar y comparar autómatas finitos, implementando las siguientes funcionalidades clave para el análisis de lenguajes regulares.

1.2 Funcionalidades Implementadas

Funcionalidad	Descripción
Lectura de AFs	Carga autómatas desde archivos de texto con formato específico
Detección de Tipo	Identifica automáticamente si un autómata es AFD o AFND
Conversión AFND→AFD	Aplica el algoritmo de construcción por subconjuntos (subset construction)
Minimización de AFD	Reduce el número de estados mediante el algoritmo de partición
Verificación de Equivalencia	Compara si dos autómatas aceptan el mismo lenguaje usando BFS sobre pares de estados
Visualización	Genera diagramas gráficos usando GraphViz (formato DOT → PNG)

Cuadro 1: Funcionalidades implementadas en el proyecto

1.3 Arquitectura del Proyecto

El proyecto sigue una estructura modular y organizada que facilita su compilación y ejecución:

```
Tarea1/
|-- src/
|   |-- Automata.java
|   |-- AutomataConverter.java
|   |-- AutomataEquivalenceChecker.java
|   |-- AutomataMinimizer.java
|   |-- GraphVizGenerator.java
|   |-- Main.java
|   '-- resultados/                (se crea automaticamente)
|       |-- automata1_original.dot
|       |-- automata1_original.png
|       |-- automata1_afd.dot
|       |-- automata1_afd.png
|       |-- automata1_minimizado.dot
|       |-- automata1_minimizado.png
|       '-- (mismo patron para automata2)
'-- input/
    |-- automata1.txt                (archivo por defecto 1)
    |-- automata2.txt                (archivo por defecto 2)
    '-- [otros archivos .txt]        (archivos de prueba)
```

1.4 Formato de Archivo de Entrada

Los autómatas deben definirse en archivos de texto plano ubicados en la carpeta `input/` con la siguiente estructura:

```
K={q0,q1,q2}          # Conjunto de estados
Sigma={a,b}            # Alfabeto de entrada
delta:                 # Inicio de definicion de transiciones
(q0,a,q0)              # Transicion: (origen, simbolo, destino)
(q0,b,q1)
(q1,a,q1)
(q1,b,q2)
(q2,a,q2)
(q2,b,q0)
s=q0                   # Estado inicial
F={q1}                # Conjunto de estados finales
```

Listing 1: Ejemplo de archivo de entrada (AFD)

Nota importante: Para AFND se permiten transiciones épsilon usando el símbolo especial `.eps`. Por ejemplo: `(q0,eps,q2)` representa una transición épsilon desde `q0` hacia `q2`.

2 Guía Rápida de Uso

2.1 Pasos para Ejecutar el Programa

Paso 1: Preparar archivos de entrada

- Coloca tus archivos `.txt` con la definición de autómatas en la carpeta `Tarea1/input/`
- Asegúrate de que sigan el formato especificado en la sección anterior

Paso 2: Abrir terminal

- Navega hasta el directorio principal del proyecto:

```
cd ruta/hacia/Tarea1
```

Paso 3: Entrar a la carpeta de código fuente

```
cd src
```

Paso 4: Compilar el proyecto

```
javac *.java
```

Paso 5: Ejecutar el programa

Opción A: Usar archivos por defecto (`automata1.txt` y `automata2.txt`)

```
java Main
```

Opción B: Especificar archivos personalizados

```
java Main archivo1.txt archivo2.txt
```

Importante: Solo escribe el nombre del archivo (ej: `caso1.txt`), NO la ruta completa. El programa buscará automáticamente en la carpeta `input/`.

Paso 6: Ver resultados

- Los archivos `.dot` y `.png` se generarán en `src/resultados/`
- La salida del programa aparecerá en la consola

2.2 Ejemplo Completo de Ejecución

```
# 1. Navegar al proyecto
cd ~/Documentos/Tarea1

# 2. Entrar a src
cd src
```

```
# 3. Compilar
javac *.java

# 4. Ejecutar con archivos por defecto
java Main

# 0 ejecutar con archivos especificos
java Main caso1a.txt caso1b.txt

# 5. Ver los resultados generados
ls resultados/
```

Listing 2: Sesión completa de terminal

3 Requisitos del Sistema

3.1 Software Necesario

Componente	Versión Mínima	Propósito
Java JDK	8 o superior	Compilar y ejecutar el código Java
GraphViz	2.40 o superior	Generar visualizaciones PNG (opcional)

Cuadro 2: Requisitos de software

3.2 Instalación de GraphViz

Descargar: <https://graphviz.org/download/>

Verificar instalación:

```
dot -V
```

Si el comando anterior muestra la versión de GraphViz, está correctamente instalado.

Nota: GraphViz es *opcional*. El programa funcionará sin él, pero no generará las imágenes PNG. Los archivos `.dot` se crearán de todas formas y pueden convertirse manualmente después.

4 Ejemplos de Uso

4.1 Ejemplo 1: Dos AFD Equivalentes

4.1.1 Archivos de Entrada

Archivo: `input/automata1.txt`

```
K={q0,q1}
Sigma={a,b}
delta:
(q0,a,q1)
(q0,b,q0)
(q1,a,q1)
(q1,b,q0)
s=q0
F={q1}
```

Archivo: input/automata2.txt

```
K={p0,p1,p2}
Sigma={a,b}
delta:
(p0,a,p1)
(p0,b,p0)
(p1,a,p1)
(p1,b,p2)
(p2,a,p1)
(p2,b,p0)
s=p0
F={p1}
```

4.1.2. Ejecución

```
cd Tarea1/src
javac *.java
java Main
```

4.1.3. Salida Esperada

```
=== Verificando archivos de entrada ===
Archivo 1: ../input/automata1.txt - OK
Archivo 2: ../input/automata2.txt - OK

=== Paso 1: Leyendo Automatas ===
Automata 1: AFD
Automata 2: AFD

--- Detalles del Automata 1 ---
Estados: [q0, q1]
Alfabeto: [a, b]
Estado inicial: q0
Estados finales: [q1]

--- Detalles del Automata 2 ---
Estados: [p0, p1, p2]
```

```
Alfabeto: [a, b]
Estado inicial: p0
Estados finales: [p1]

=== Generando diagramas de los automatas originales ===
Directorio 'resultados/' creado.
Archivo DOT generado: resultados/automata1_original.dot
Imagen generada: resultados/automata1_original.png
Archivo DOT generado: resultados/automata2_original.dot
Imagen generada: resultados/automata2_original.png

=== Paso 2: Convirtiendo a AFD ===
Automata 1 ya es AFD (no requiere conversion)
Automata 2 ya es AFD (no requiere conversion)

=== Generando diagramas de los AFD convertidos ===
Archivo DOT generado: resultados/automata1_afd.dot
Imagen generada: resultados/automata1_afd.png
Archivo DOT generado: resultados/automata2_afd.dot
Imagen generada: resultados/automata2_afd.png

=== Paso 3: Minimizando y Verificando Equivalencia ===

=====
RESULTADO: Los automatas son EQUIVALENTES
Ambos automatas aceptan el mismo lenguaje.
=====

=== Generando diagramas de los automatas minimizados ===
Archivo DOT generado: resultados/automata1_minimizado.dot
Imagen generada: resultados/automata1_minimizado.png
Archivo DOT generado: resultados/automata2_minimizado.dot
Imagen generada: resultados/automata2_minimizado.png

=== Proceso Completado ===
Archivos generados (3 versiones por automata):
  Automata 1:
    - resultados/automata1_original.dot/.png
    - resultados/automata1_afd.dot/.png
    - resultados/automata1_minimizado.dot/.png
  Automata 2:
    - resultados/automata2_original.dot/.png
    - resultados/automata2_afd.dot/.png
    - resultados/automata2_minimizado.dot/.png
```

4.2 Ejemplo 2: Conversión de AFND a AFD

4.2.1. Archivos de Entrada

Archivo input/afnd_ejemplo.txt (AFND con transiciones epsilon):

```
K={q0,q1,q2}
Sigma={a,b}
```



```
delta:
(q0,a,q0)
(q0,eps,q1)
(q1,b,q2)
(q2,a,q2)
(q2,b,q0)
s=q0
F={q2}
```

Archivo input/afd_equivalente.txt (AFD equivalente):

```
K={p0,p1}
Sigma={a,b}
delta:
(p0,a,p0)
(p0,b,p1)
(p1,a,p1)
(p1,b,p0)
s=p0
F={p1}
```

4.2.2. Ejecución

```
java Main afnd_ejemplo.txt afd_equivalente.txt
```

4.2.3. Salida Esperada (Resumen)

```
=== Paso 1: Leyendo Automatas ===
Automata 1: AFND
Automata 2: AFD

=== Generando diagramas de los automatas originales ===
[Generacion exitosa...]

=== Paso 2: Convirtiendo a AFD ===
Convirtiendo Automata 1 de AFND a AFD...
Conversion completada.
Automata 2 ya es AFD (no requiere conversion)

Despues de la conversion:
Automata 1: AFD
Automata 2: AFD

=== Paso 3: Minimizando y Verificando Equivalencia ===

=====
RESULTADO: Los automatas son EQUIVALENTES
=====
```

4.3 Ejemplo 3: Autómatas No Equivalentes

4.3.1. Archivos de Entrada

Archivo input/aut01.txt

```
K={q0,q1}
Sigma={a}
delta:
(q0,a,q1)
(q1,a,q1)
s=q0
F={q1}
```

Archivo input/aut02.txt

```
K={p0}
Sigma={a}
delta:
(p0,a,p0)
s=p0
F={p0}
```

4.3.2. Ejecución

```
java Main aut01.txt aut02.txt
```

4.3.3. Salida Esperada

```
...
=== Paso 3: Minimizando y Verificando Equivalencia ===

=====
RESULTADO: Los automatas NO son EQUIVALENTES
Los automatas aceptan lenguajes diferentes.
=====
```

5 Algoritmos Implementados

5.1 Conversión AFND $\rightarrow$ AFD (Subset Construction)

El algoritmo de construcción por subconjuntos transforma un autómata finito no determinista en uno determinista equivalente.

Pasos principales:

1. Calcular la ϵ -clausura del estado inicial para obtener el estado inicial del AFD.

2. Para cada subconjunto de estados (que representa un estado del AFD) y cada símbolo del alfabeto:
 - a. Calcular $\text{move}(S, \text{símbolo}) = \text{conjunto de estados alcanzables desde } S \text{ con ese símbolo}$.
 - b. Calcular la ϵ -clausura del resultado.
 - c. Si este subconjunto no existe en el AFD, crearlo como nuevo estado.
 - d. Agregar la transición correspondiente.
3. Un estado del AFD es final si su subconjunto contiene al menos un estado final del AFND.
4. Agregar un estado *trampa* para completar todas las transiciones faltantes (hacer el AFD completo).

5.2 Minimización de AFD (Algoritmo de Partición)

El algoritmo de minimización reduce el número de estados eliminando estados equivalentes.

Pasos principales:

1. **Eliminar estados inalcanzables:** Realizar BFS desde el estado inicial para identificar estados alcanzables. Eliminar los demás.
2. **Partición inicial:** Crear dos grupos: estados finales y estados no finales.
3. **Refinamiento iterativo:**
 - Para cada grupo de la partición actual, verificar si todos los estados son equivalentes.
 - Dos estados son equivalentes si para cada símbolo del alfabeto, sus transiciones llevan a estados del mismo grupo.
 - Si se encuentran estados no equivalentes en un grupo, dividir el grupo.
 - Repetir hasta que no se puedan hacer más divisiones.
4. **Construcción del AFD minimizado:** Cada grupo final de equivalencia se convierte en un estado del nuevo autómata.

5.3 Verificación de Equivalencia (BFS sobre pares)

Determina si dos AFD aceptan el mismo lenguaje explorando pares de estados simultáneamente.

Pasos principales:

1. Verificar que ambos autómatas tengan el mismo alfabeto. Si no, no son equivalentes.
2. Inicializar una cola con el par de estados iniciales: $(\text{inicial}_1, \text{inicial}_2)$.

3. Mientras la cola no esté vacía:
 - a) Extraer un par (q_1, q_2) de la cola.
 - b) Verificar que ambos sean finales o ambos no finales. Si difieren, los autómatas no son equivalentes.
 - c) Para cada símbolo a del alfabeto:
 - Obtener $q'_1 = \delta_1(q_1, a)$ y $q'_2 = \delta_2(q_2, a)$
 - Si el par (q'_1, q'_2) no ha sido visitado, agregarlo a la cola.
4. Si se procesaron todos los pares sin encontrar diferencias, los autómatas son equivalentes.

6 Visualización con GraphViz

6.1 Archivos Generados

Para cada autómata procesado se generan automáticamente **tres versiones visuales**:

Archivo	Descripción
automata1_original.dot/.png	Autómata tal como se leyó del archivo de entrada
automata1_afd.dot/.png	Autómata convertido a AFD (sin minimizar)
automata1_minimizado.dot/.png	Autómata minimizado (versión final optimizada)

Cuadro 3: Archivos generados por autómata (el mismo patrón se aplica a automata2)

Todos estos archivos se encuentran en el directorio `src/resultados/`.

6.2 Formato DOT

El formato DOT es un lenguaje de descripción de grafos utilizado por GraphViz. Ejemplo de un archivo generado:

```
digraph Automata {
    rankdir=LR;
    "" [shape=none];
    "M{q1}" [shape=doublecircle];
    node [shape=circle];
    "" -> "M{q0}";
    "M{q0}" -> "M{q1}" [label="a"];
    "M{q0}" -> "M{q0}" [label="b"];
    "M{q1}" -> "M{q1}" [label="a"];
    "M{q1}" -> "M{q0}" [label="b"];
}
```

Listing 3: Ejemplo de automata1_minimizado.dot

6.3 Generación Manual de Imágenes

Si GraphViz está instalado pero las imágenes no se generaron automáticamente, puede crearlas manualmente:

```
cd src/resultados

# Generar imagenes para automata 1
dot -Tpng automata1_original.dot -o automata1_original.png
dot -Tpng automata1_afd.dot -o automata1_afd.png
dot -Tpng automata1_minimizado.dot -o automata1_minimizado.png

# Generar imagenes para automata 2
dot -Tpng automata2_original.dot -o automata2_original.png
dot -Tpng automata2_afd.dot -o automata2_afd.png
dot -Tpng automata2_minimizado.dot -o automata2_minimizado.png
```